

Introducere în programare - curs și laborator - suport electronic

(C) 2024-2025 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#)

Lectia 3 - Enumerări, structuri, tablouri și pointeri

În acest curs vor fi prezentate tipurile de date enumerare, tablouri, structuri. Vor fi prezentate unele lucruri mai puțin cunoscute privind reprezentarea tablourilor, parcurgerea lor, funcții cu tablouri ca argumente. De asemenea, se vor explica câmpurile de biți și legătura dintre pointeri și tablouri.

Rezumat prima parte



Rezumat partea a doua



Video

- Metoda biseției -: pentru exercitiul A din 3.1/laboratorul 3. (7:36): [click aici](#)
- Calculul rădăcinii pătrate -: pentru exercițiul B din 3.1/laboratorul 3. (9:55): [click aici](#)
- Enumerări, structuri, tablouri și pointeri (15:25): [click aici](#)
- Exemple cu struct și tablouri (16:08): [click aici](#)
- Reprezentarea tablourilor (7:31): [click aici](#)
- Înmulțirea matricelor (6:41): [click aici](#)
- Sugestie de reprezentare a datelor pentru jocuri pe tablă (19:22): [click aici](#)

3.1. Enumerări

Există situații în practica în care un program trebuie să prelucreze variabile de tip întreg care au câteva valori discrete cărora li se atribuie o anumită semnificație.

▼ Exemplu

Spre exemplu, un program care prelucrează date calendaristice va trebui să poată prelucra variabile care semnifică zilele săptămânii. În acest caz, se poate stabili o convenție prin care fiecărei zile din săptămână i se atribuie o valoare întreagă (spre exemplu ziua de luni se reprezintă prin valoarea 0, ziua de duminică se reprezintă prin valoarea 6).

ATENȚIE! Codul acesta nu va funcționa în C++, ci doar în C.

```
int main ()
{
    int azi = 3; /* joi */
    int ieri = azi - 1;
    return 0;
}
```

Dificultatea constă însă în memorarea de către programator a valorilor asociate fiecărei zile din săptămână. O prima soluție ar fi utilizarea unor constante declarate astfel:

```
const int luni = 0; const int marti = 1; const int miercuri = 2; const int joi = 3;
const int vineri = 4; const int sambata = 5; const int duminica = 6;
int main () {
    int azi = joi; /* joi */
    int ieri = azi - 1;
    return 0;
}
```

Această soluție îmbunătățește claritatea programului, dar presupune un oarecare efort și atenție sporită pentru declararea constantelor `luni ... duminica`.

În situații de tipul acesta, pentru a spori claritatea programelor, limbajul C oferă posibilitatea de a defini o **enumerare**. Limbajul C permite definirea unui set de constante numerice întregi între care există o legătură din punctul de vedere al programatorului utilizând o declarație de tip `enum`.

ATENȚIE! Codul acesta nu va funcționa în C++, ci doar în C.

▼ Exemple

```
enum zile { luni, marti, miercuri, joi, vineri, sambata, duminica };
int main ()
{
    enum zile azi; enum zile ieri; ...
    azi = joi; ieri = azi - 1;
    return 0;
}

enum culori { rosu, albastru, verde, violet, galben, portocaliu, maro };
int main ()
{
    enum culori culoare1; enum culori culoare2; ...
    culoare1 = verde; culoare2 = culoare1 + 2;
    return 0;
}
```

Sintaxa completă pentru declararea unei enumerări este următoarea:

```
enum nume_enumerare {
    constanta1 = valoare1,
    constanta2 = valoare2,
    ...,
    constantaN = valoareN
} variabila1, ...,variabilaN;
```

În aceasta declarație, `nume_enumerare` este opțional, dar dacă se specifică, se pot crea ulterior și alte variabile întregi asociate acestui tip (`enum zile azi; enum zile ieri`). De asemenea, valorile `valoare1 ... valoareN`, asociate fiecărei constante sunt opționale. Dacă nu se specifică explicit, se aplică următoarea regulă:

- pentru prima constantă, dacă nu se specifică explicit valoarea, i se atribuie valoarea 0;
- pentru celelalte constante, dacă nu au specificată explicit valoarea dorită, li se atribuie valoarea constantei precedente + 1

Așadar variabilele declarate pe baza unei enumerări sunt variabile de tip întreg. Ulterior, în program, simbolurile `constant1 ... constantN` se tratează ca orice constante cu valorile stabilite pe baza regulii precedente și se pot atribui oricăror variabile de tip întreg.

▼ Exemplu

Ele sunt asociate enumerării `zile` doar pentru a indica intenția programatorului de a folosi aceste variabile pentru a reprezenta zilele săptămânii, îmbunătățind claritatea programului.

Odată cu declararea unei enumerări, opțional, se pot declara și variabile care să fie asociate acelei enumerări (`variabila1 ... variabilaN`).

▼ Exemplu

```
enum gen_substantiv {
    neutru, /* valoarea implicită 0 */
    feminin = 12, /* valoarea explicită 12 */
    masculin /* valoarea implicită 13 */
} gen1, gen2; /* 2 variabile întregi */
```

Evident, aceeași constantă (simbol) nu poate să apară în mai multe declarații de enumerări: `enum zile { luni, marti, miercuri, joi, vineri, sambata, duminica }; /* eroare, constantele sambata și duminica au fost definite deja */ enum zile_weekend { sambata, duminica };`

▼ Exemple

```
enum culori {
    rosu, /* valoare implicită 0 */
    verde, /* valoare implicită 1 */
    albastru /* valoare implicită 2 */
} c; /* variabila de tip întreg */
enum zile {
    luni = 1, /* valoare explicită 1 */
    marti, /* valoare implicită 2 */
    miercuri, joi, vineri, sambata,
    duminica /* ajunge la valoarea 7 */
} ieri, azi, maine;
int main ()
{
    c = rosu; azi = joi; ieri = azi - 1; maine = azi + 1;
    printf ("Azi este: %d \n", azi); /* afiseaza 4 */
    char sir [80];
    switch (c)
    {
        case rosu: strcpy (sir, "rosu"); break;
        case verde: strcpy (sir, "verde");break;
        case albastru: strcpy (sir, "albastru"); break;
        default: strcpy (sir, "Eroare, contactați programatorul !");
    }
    printf ("Culoarea: %s \n", sir); /*afişează "rosu" */
    return 0;
}
```

3.2. Tablouri

Tablourile reprezintă o colecție de variabile de același tip, apelate cu același nume. Componentele tabloului sunt identificate și accesate cu ajutorul indicilor. Un tablou ocupă locații de memorie contigue, în ordinea indicilor. Tablourile pot fi: unidimensionale (1 – dimensionale, vectori, șiruri) (un caz special constituindu-l șirurile de caractere), bidimensionale (2 - dimensionale) sau cu mai multe dimensiuni.

3.2.1. Tablouri unidimensionale

Acestea se declară astfel:

```
tip numeTablou[dimensiune];
```

Numărul de octeți necesari reprezentării unui tablou unidimensional este = `sizeof(tip) * dimensiune`.

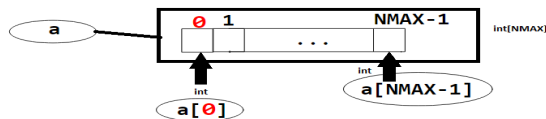
▼ Exemplu

```
#define NMAX 25
int a[NMAX]; // int a[25];
```

▼ Reprezentare

```
tip numeTablou[dimensiune];
octeți_necesari = sizeof(tip) * dimensiune
```

```
#define NMAX 25
int a[NMAX]; // int a[25];
```



Ordinea de memorare a elementelor din tablou este ordinea indicilor: 0, 1, ..., NMAX-1, elementele tabloului fiind în zone de memorie contigue (una lângă alta); La un tablou, operațiile (de citire, scriere, actualizare) se realizează prin intermediul componentelor, folosind iterații `for`, `while` sau `do-while`:

▼ Exemplu

```
for (i=0; i<n; i++) a[i]=0;
for (i=0; i<n; i++) c[i]=a[i]+ b[i];
```

- Indexarea tablourilor începe de la **0** până la **(dim-1)**
- Dacă se depășește limita tabloului, apar rezultate neașteptate

```
for (count = 1 ; count <=
NO_OF_STUDS ; count++)
{cout << "Orele petrecute ...cu
numărul: " << count << " ";
cin >> hours[count];
}
```

hours	
?	hours[0]
38	hours[1]
42	hours[2]
29	hours[3]
35	hours[4]
38	hours[5]

Numele unui tablou reprezintă atât un nume de variabilă, cât și un **pointer către primul element din tablou**. Astfel, avem următoarele echivalențe:

- `a` este echivalent cu `&a[0]`;
- `*a` este echivalent cu `a[0]`;
- `a+1` este echivalent cu `&a[1]`;
- `*(a+1)` este echivalent cu `a[1]`;
- `a+2` este echivalent cu `&a[2]`;
- `*(a+2)` este echivalent cu `a[2]`;
- `a+i` este echivalent cu `&a[i]`;
- `*(a+i)` este echivalent cu `a[i]`.

▼ Exemple de inițializare a unui tablou unidimensional

- Indexarea tablourilor începe de la **0** până la **(dim-1)**
- Dacă se depășește limita tabloului, apar rezultate neașteptate

```
for (count = 1 ; count <=
NO_OF_STUDS ; count++)
{cout << "Orele petrecute ...cu
numărul: " << count << ": ";
cin >> hours[count];
}
```

hours	
?	hours[0]
38	hours[1]
42	hours[2]
29	hours[3]
35	hours[4]
38	hours[5]

```
int matr[4];

matr[0] = 6;
matr[1] = 0;
matr[2] = 9;
matr[3] = 6;
```

```
int matr[4] = { 6, 0, 9, 6 };
```

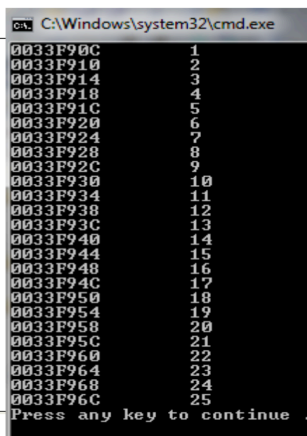
```
int matr[] = { 6, 0, 9, 6, 5, -9, 0 };
```

▼ Exemplu de memorare a unui vector

Memorarea unui tablou

```
#include <iostream>
using namespace std;

#define NMAX 25
int main(){
    int a[NMAX], i;
    for (i=0; i<NMAX; i++){
        a[i] = i+1;
        cout << &a[i] << "\t" <<
            *(a+i)<< "\n";
    }
    return 0;
}
```



```
C:\Windows\system32\cmd.exe
0033F90C      1
0033F910      2
0033F914      3
0033F918      4
0033F91C      5
0033F920      6
0033F924      7
0033F928      8
0033F92C      9
0033F930     10
0033F934     11
0033F938     12
0033F93C     13
0033F940     14
0033F944     15
0033F948     16
0033F94C     17
0033F950     18
0033F954     19
0033F958     20
0033F95C     21
0033F960     22
0033F964     23
0033F968     24
0033F96C     25
Press any key to continue .
```

▼ Variante de parcurgere a unui vector

Avem un vector (tablou unidimensional) **a** de numere. Dorim să calculăm suma elementelor sale, după ce am inițializat **suma** cu 0. Avem mai multe modalități de a parcurge tabloul, care țin de legătura dintre tablouri și pointeri.

```
/* Varianta 1 */ for (i=0; i < n; ++i)
                 suma += a[i];
```

```
/* Varianta 2 */ for (i=0; i < n; ++i)
                 suma += *(a+i);
```

```
/* Varianta 3 */ for (p=a; p < &a[n]; ++p)
                 suma += *p;
```

```
/* Varianta 4 */ p=a;
                 for (i=0; i < n; ++i)
                     suma += p[i];
```

Tablourile ca argumente ale funcțiilor

- Un tablou NU poate fi transmis ÎN ÎNTREGIME ca argument al unei funcții.
- Tablourile se transmit ca argumente folosind un pointer la un tablou.
- Parametrul formal al unei funcții ce primește un tablou poate fi declarat ca: pointer, tablou cu dimensiune, tablou fără dimensiune

▼ Exemplul 1

Să considerăm că avem un tablou `i` și apelăm o funcție `functia` cu argumentul `i`.

```
int main(void){
    int i[4];
    functia(i);
    return 0;
}
```

În acest caz, `functia` va putea avea una din următoarele antete, în care argumentul poate fi: pointer la un număr întreg, un tablou cu un număr specificat de elemente întregi, un tablou cu un număr nespecificat de elemente întregi.

```
void functia(int *x)
```

```
void functia(int tab[4])
```

```
void functia(int tab[])
```

▼ Exemplul 2

În următorul exemplu, funcția `insert_sort` are ca argument un tablou cu un număr nespecificat de elemente întregi. Deoarece argumentul `a[]` este echivalent cu pointerul `*a`, funcția va modifica elementele tabloului `a`, realizând, de pildă,

```
void insert_sort(int a[], int n)
{
    //...
}
```

```
/* utilizare */
int w[100];
// ...
insert_sort(w, 10);
```

sortarea prin inserție a elementelor vectorului `a`, având `n` elemente.

▼ Exemplul 3

În acest exemplu, presupunem ca funcția `suma` dorește să realizeze suma celor `n` elemente din tabloul `a`.

- Putem avea antetul funcției în două feluri, în care primul argument este fie `a[]`, fie `*a`, deci un pointer la începutul

```
double suma(double a[], int n);
// double suma(double *a, int n);
```

```
suma(v, 100);
suma(v, 8);
suma(&v[4], k-6);
suma(v+4, k-6);
```

tabloului.

- În primul apel, `suma(v,100)`, funcția va calcula suma celor 100 elemente din tablou, începând cu elementul de pe poziția 0 (și terminând cu elementul de pe poziția 99).
- În al doilea apel, `suma(v,8)`, se va calcula suma `v[0]+v[1]+...+v[7]`.
- În al treilea apel, primul argument este adresa elementului de pe poziția 4 din tablou, deci suma care va fi calculată va fi `v[4]+v[5]+... + v[k-6-1+4]` (total `k-6` elemente).
- În al patrulea apel, primul argument (`v+4`) este echivalent cu argumentul `&v[4]`, reprezentând faptul că se calculează tot `v[4]+v[5]+...`.

3.2.2. Șiruri de caractere

Șirurile de caractere sunt tablouri unidimensionale de tip `char`. Fiecare element al tabloului este un caracter. Ultimul caracter al șirului este caracterul nul `'\0'`. Acesta marchează sfârșitul șirului. Exemplu:

```
char sir[10];
```

Aceasta este o declarație pentru un șir de 9 caractere (`sir[0]`, `sir[1]`..., `sir[8]`), la care se adaugă `sir[9]`, ce va marca sfârșitul șirului. Al 10-lea caracter va fi caracterul nul `'\0'` (sau `NULL` sau `0`). Un **șir de caractere** ("*string*") este o zonă de memorie ocupată cu caractere/"char"-uri (un "char" ocupă un octet), terminată cu un octet de valoare 0 (deoarece caracterul `'\0'` are codul ASCII egal cu 0). O variabilă care reprezintă un șir de caractere este un pointer (reține adresa) la primul octet.

Un șir de caractere se poate reprezenta ca:

- tablou de caractere (pointer constant):
 - `char sir1[10];`
 - `char sir2[10] = "exemplu";`
- pointer la caractere:
 - `char *sir3;`
 - `char *sir4 = "exemplu";`

▼ Exemplu declarare șir de caractere

```
#define MAX_SIR 100 // suficient de mare
...
char sirCaract[MAX_SIR];
```

▼ Declarare șiruri cu inițializare

```
char s[] = "Hi Mom!";

char s[10]="Hi Mom!";

char s[10]= {'H', 'i', ' ', 'M', 'o', 'm', '!', '\0'};
```

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
H	i		M	o	m	!	\0	?	?

Asignarea șirurilor de caractere (=) se poate face doar la declarare:

```
char sir[10];
sir = "Hello"; // gresit
char sir[10] = "Hello"; // corect
```

Pentru a copia conținutul unui șir în alt șir, se folosește funcția `strcpy`.

```
char sir[10] = "Hello";
char s[10], t[10];
strcpy(s, sir);
strcpy(t, "Buna");
```

Compararea între șiruri de caractere nu e posibilă prin operatorul `==`

```
char sir_unu[10] = "alba";
char sir_doi[10] = "neagra";
sir_unu == sir_doi; // warning: operator has no effect
```

Pentru comparare se folosește o funcție specială, numită `strcmp`. Apelul `strcmp(s, t)` va returna 0, dacă `s` și `t` sunt identice, un număr `<0`, dacă `s` este lexicografic înaintea lui `t`, respectiv un număr `>0`, dacă `s` este lexicografic după `t`.

Șirurile de caractere din C pot conține caractere din cele 256 din lista ASCII (American Standard Code for Information Interchange). Comparațiile între șirurile de caractere se fac lexicografic, pe baza codurilor ASCII ale caracterelor, de la stânga la dreapta.

▼ Codurile ASCII de bază (0-127)

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	Space	64	40	100	64	0	96	60	140	96	0	140
1	1	001	SOH (start of heading)	33	21	041	!	65	41	101	65	A	97	61	141	97	A	141
2	2	002	STX (start of text)	34	22	042	"	66	42	102	66	B	98	62	142	98	B	142
3	3	003	ETX (end of text)	35	23	043	#	67	43	103	67	C	99	63	143	99	C	143
4	4	004	EOF (end of transmission)	36	24	044	\$	68	44	104	68	D	100	64	144	100	D	144
5	5	005	ENQ (enquiry)	37	25	045	%	69	45	105	69	E	101	65	145	101	E	145
6	6	006	ACK (acknowledge)	38	26	046	&	70	46	106	70	F	102	66	146	102	F	146
7	7	007	BEL (bell)	39	27	047	'	71	47	107	71	G	103	67	147	103	G	147
8	8	010	BS (backspace)	40	28	050	(	72	48	110	72	H	104	68	150	104	H	150
9	9	011	TAB (horizontal tab)	41	29	051	)	73	49	111	73	I	105	69	151	105	I	151
10	A	012	LF (NL line feed, new line)	42	2A	052	*	74	4A	112	74	J	106	6A	152	106	J	152
11	B	013	VT (vertical tab)	43	2B	053	+	75	4B	113	75	K	107	6B	153	107	K	153
12	C	014	FF (NP form feed, new page)	44	2C	054	,	76	4C	114	76	L	108	6C	154	108	L	154
13	D	015	CR (carriage return)	45	2D	055	-	77	4D	115	77	M	109	6D	155	109	M	155
14	E	016	SO (shift out)	46	2E	056	.	78	4E	116	78	N	110	6E	156	110	N	156
15	F	017	SI (shift in)	47	2F	057	/	79	4F	117	79	O	111	6F	157	111	O	157
16	10	020	DLE (data link escape)	48	30	060	0	80	50	120	80	P	112	70	160	112	P	160
17	11	021	DC1 (device control 1)	49	31	061	1	81	51	121	81	Q	113	71	161	113	Q	161
18	12	022	DC2 (device control 2)	50	32	062	2	82	52	122	82	R	114	72	162	114	R	162
19	13	023	DC3 (device control 3)	51	33	063	3	83	53	123	83	S	115	73	163	115	S	163
20	14	024	DC4 (device control 4)	52	34	064	4	84	54	124	84	T	116	74	164	116	T	164
21	15	025	NAK (negative acknowledge)	53	35	065	5	85	55	125	85	U	117	75	165	117	U	165
22	16	026	SYN (synchronous idle)	54	36	066	6	86	56	126	86	V	118	76	166	118	V	166
23	17	027	ETB (end of trans. block)	55	37	067	7	87	57	127	87	W	119	77	167	119	W	167
24	18	030	CAN (cancel)	56	38	070	8	88	58	130	88	X	120	78	170	120	X	170
25	19	031	EM (end of medium)	57	39	071	9	89	59	131	89	Y	121	79	171	121	Y	171
26	1A	032	SUB (substitute)	58	3A	072	:	90	5A	132	90	Z	122	7A	172	122	Z	172
27	1B	033	ESC (escape)	59	3B	073	;	91	5B	133	91	[	123	7B	173	123	[	173
28	1C	034	FS (file separator)	60	3C	074	<	92	5C	134	92	\	124	7C	174	124	\	174
29	1D	035	GS (group separator)	61	3D	075	=	93	5D	135	93	]	125	7D	175	125	]	175
30	1E	036	RS (record separator)	62	3E	076	>	94	5E	136	94	^	126	7E	176	126	^	176
31	1F	037	US (unit separator)	63	3F	077	?	95	5F	137	95	_	127	7F	177	127	_	177

▼ Coduri ASCII extinse (128-255)

128	Ç	144	É	160	á	176	ð	192	Ł	208	ł	224	α	240	≡
129	ü	145	æ	161	í	177	ñ	193	Ł	209	ł	225	β	241	±
130	é	146	Æ	162	ó	178	■	194	Ŧ	210	ŧ	226	Γ	242	≥
131	â	147	ô	163	ú	179		195	Ŧ	211	ŧ	227	π	243	≤
132	ä	148	ö	164	ñ	180	†	196	—	212	ℓ	228	Σ	244	∫
133	à	149	ò	165	Ñ	181	‡	197	†	213	ℓ	229	σ	245	∫
134	â	150	û	166	•	182	‡	198	†	214	ℓ	230	μ	246	+
135	ç	151	ù	167	°	183	¶	199	‡	215	‡	231	τ	247	≈
136	ê	152	ÿ	168	¿	184	¶	200	ℓ	216	‡	232	Φ	248	°
137	ë	153	Ö	169	¬	185	¶	201	ℓ	217	‡	233	Θ	249	·
138	è	154	Ü	170	¬	186	¶	202	ℓ	218	‡	234	Ω	250	·
139	ï	155	•	171	½	187	¶	203	¶	219	■	235	δ	251	√
140	î	156	£	172	¾	188	¶	204	‡	220	■	236	∞	252	∞
141	ï	157	¥	173	ı	189	¶	205	=	221	■	237	φ	253	²
142	Ä	158	£	174	«	190	¶	206	‡	222	■	238	e	254	■
143	Å	159	f	175	»	191	¶	207	±	223	■	239	∩	255	

Așadar, **macrodefinițiile și funcțiile pentru șiruri** se folosesc pentru a face diferite operații pe șiruri de caractere. De pildă, **strcat** permite concatenarea unui șir la un alt șir, **strlen()** returnează lungimea unui șir (numărul de caractere, fără '\0') etc. Vom reveni asupra lor în cursul următor.

▼ Câteva macrodefiniții

• In fisierul **<ctype>** (**ctype.h**) (manipularea caracterelor)
isspace(c), **isdigit(c)**, **islower(c)**, **tolower(c)**, **toupper(c)**,
 ...

• In fisierul **<string>** (**string.h**) (manipularea șirurilor)
char * strcat (char * destination, const char * source);

int strcmp(const char *s1,const char*s2);

char * strcpy (char * destination, const char * source);

size_t strlen (const char * str);
 char mystr[100]="test string";
 sizeof(mystr); // 100
 strlen(mystr); // 11

const char * strchr (const char * str, int character);
char * strchr (char * str, int character);

3.2.3. Tablouri bidimensionale

Declararea unui tablou bidimensional (matrice) se face astfel:

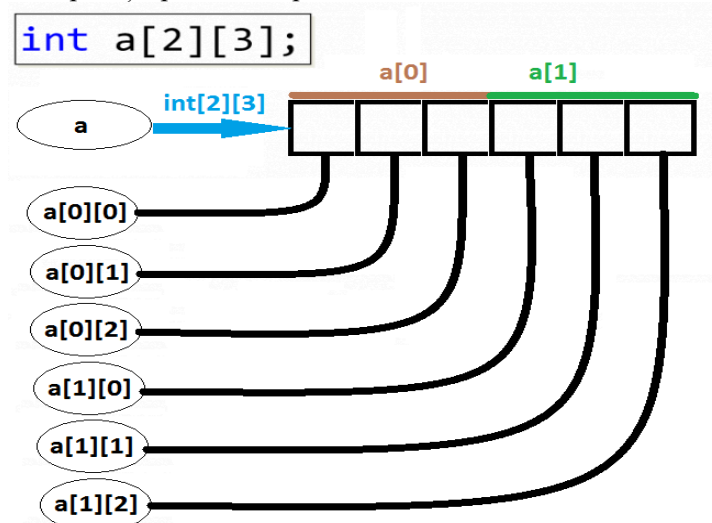
```
tip numeTablou[m][n];
int a[m][n];
```


Matricea ocupă o memorie contiguă de $m \times n$ locații, iar componentele sunt identificate prin doi indici:

- primul indice are valori $\{0, 1, \dots, m-1\}$
- al doilea indice are valori $\{0, 1, \dots, n-1\}$

Variabilele componente ale acestei matrice sunt: $a[0][0], a[0][1], \dots, a[0][n-1], a[1][0], a[1][1], \dots, a[1][n-1], \dots, a[m-1][0], a[m-1][1], \dots, a[m-1][n-1]$ Ordinea de memorare a componentelor este dată de ordinea lexicografică a indicilor

▼ Explicație pe un exemplu



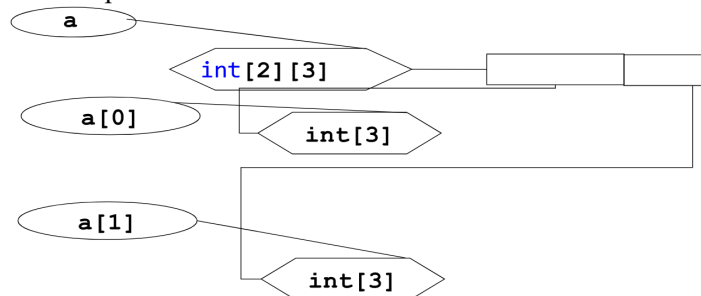
În figura de mai sus, a este de tip $\text{int}[2][3]$, $a[0]$ este de tip $\text{int}[3]$, $a[1]$ este de tip $\text{int}[3]$, iar fiecare element din matrice este de tip int .

Parcurgerea unui tablou bidimensional se poate realiza după cum urmează:

```
double a[MMAX][NMAX]; // declarare tablou bidimensional (matrice de numere reale în precizie dublă)
double suma; // suma elementelor din tablou
/* citirea elementelor, prin parcurgerea după linii, coloane */
for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
        cin >> a[i][j];
/* suma tuturor elementelor din tablou */
suma = 0;
for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
        suma += a[i][j];
```

Un tablou bidimensional poate fi privit ca un tablou unidimensional în care fiecare componentă este un tablou unidimensional.

▼ Exemplu



▼ Accesarea elementelor unui tablou bidimensional

	coloana 0	coloana 1	...
linia 0	$a[0][0]$	$a[0][1]$	...
linia 1	$a[1][0]$	$a[1][1]$	...
...	...	...	...

Ca și în cazul tablourilor unidimensionale, există **expresii echivalente cu $a[i][j]$** , care fac legătura dintre tablouri și pointeri:

- `* (a[i]+j),`
- `* (* (a+i)+j),`
- `(* (a+i)) [j],`
- `* (&a[0][0]+NMAX*i+j) .`

Tablouri bidimensionale ca argumente de funcție:

```
int minmax(int t[][NMAX], int i0, int j0, int m, int n)
{
    //...
}
/* utilizare */
if (minmax(a,i,j,m,n))
{
    // ...
}
```

Inițializarea tablourilor (vectori, siruri, matrice)

```
int a[] = {-1, 0, 4, 7};
/* echivalent cu */
int a[4] = {-1, 0, 4, 7};
char s[] = "un sir";
/* echivalent cu */
char s[7] = {'u', 'n', ' ', 's', 'i', 'r', '\0'};
int b[2][3] = {1,2,3,4,5,6} /* echivalent cu */
int b[2][3] = {{1,2,3},{4,5,6}} /*echivalent cu*/
int b[][3] = {{1,2,3},{4,5,6}}
```

3.3. Pointeri

1. Pointer = variabilă care conține o adresă din memorie, adresă care este localizarea unui obiect (de obicei altă variabilă)
2. Oferă posibilitatea modificării argumentelor de apelare a funcțiilor
3. Permite alocarea dinamică
4. Pot îmbunătăți eficiența unor rutine
5. Pointeri neinițializați
6. Pointeri ce conțin valori inadecvate

3.3.1. Declararea unei variabile pointer:

tip `*nume_pointer;`

- *nume_pointer* este o variabilă ce poate avea valori adrese din memorie ce conțin valori cu tipul de bază *tip*.
- Exemple:

```
int *p, i; // int *p; int i;
p = 0;
p = NULL;
p = &i;
p = (int*) 232;
```

Semnificația lui `p = &i;`

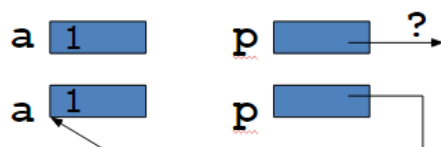
- **p** “pontează la i”, “conține / primește adresa lui i”, “referențiază la i”.

Operatorul de dereferențiere (indirectare) `*` : `int *p;`

- **p** este pointer, ***p** este / primește valoarea variabilei ce are adresa **p**

Valoarea directă a lui **p** este adresa unei locații iar ***p** este valoarea indirectă a lui **p**: ceea ce este memorat în locația respectivă

```
int a = 1, *p;
p = &a;
```



```
int a = 1, *p; p = &a;
```

- pentru că memorează adrese, lungimile locațiilor de memorie nu depind de tipul variabilei asociate

`sizeof(int*) = sizeof(double*) = ...`

- afișarea unui pointer:

```
int *px, x = 0, *py, y = 0;
px = &x; py = &y;
cout << "px= " << px << " , py = " << py << endl;
```

```
C:\Windows\system32\cmd.exe
px= 0043F8A4 , py = 0043F88C
Press any key to continue . .
```

3.3.2. Expresii cu pointeri

```
int i = 3, j = 5;
int *p = &i, *q = &j, *r;
double x;
```

Expresia	Echivalent	Valoare
<code>p = &i</code>	<code>p = (&i)</code>	002EFEA4
<code>**&p</code>	<code>*(&p)</code>	3
<code>r = &x</code>	<code>r = (&x)</code>	eroare!
	(cannot convert from double* to int*)	
<code>3**p/(**q)+2</code>	<code>((3*(*p)))/(*q))+2</code>	3
<code>*(r=&j)*=*p</code>	<code>(*(&j))*=(*p)</code>	15

```
int *i;    float *f;    void *v;
```

Expresii corecte

Expresii incorecte

<code>i = 0;</code>	<code>i = 1;</code>
<code>i = (int*)1;</code>	<code>v = 1;</code>
<code>v = f; i=(int *)v;</code>	<code>i = f;</code>
<code>i = (int*)f;</code>	<code>&3;</code>
<code>f = (float*)v;</code>	<code>&(k+8);</code>
<code>*((int*)333);</code>	<code>*333;</code>

▼ Exemplu

```
#include <iostream>
using namespace std;
int main(void){
    int i=5, *p = &i; float *q; void *v;
    q = (float*)p; v = q;
```

```

        cout << "p = " << p << ", *p = " << *p << "\n";
        cout << "q = " << q << ", *q = " << *q << "\n";
        cout << "v = " << v << ", *v = " << *((float*)v) << "\n";
        return 0;
    }

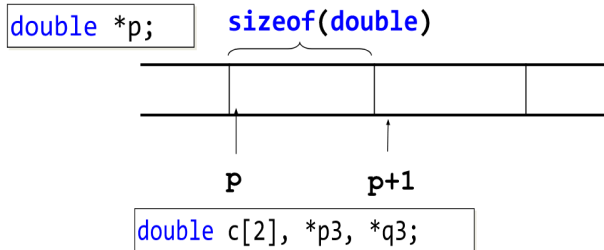
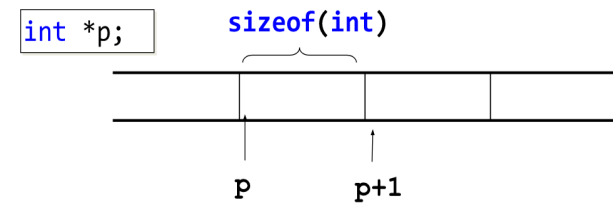
```

```

p = 0030F738, *p = 5
q = 0030F738, *q = 7.00649e-045
v = 0030F738, *v = 7.00649e-045
Press any key to continue . . .

```

3.3.3. Aritmetica pointerilor



```

p3 = c;
q3 = p3 + 1;
cout << q3-p3;
cout << sizeof(double), (int)q3 - (int)p3;

```

$q3 - p3 = 1$
 $\text{sizeof}(\text{double}) = 8, (\text{int})q3 - (\text{int})p3 = 8$

▼ Video legătura pointeri - tablouri

<https://youtu.be/ASVB8KAFypk>

3.3.4. Diferențe între pointeri și referințe

Iată o secvență de cod ce inițializează un număr folosind pointeri (C,C++):

```

void SetInt(int *i){
    (*i) = 5;
}
void main(){
    int x;
    SetInt(&x);
}

```

și o secvență de cod ce inițializează un număr folosind referințe (C++):

```

void SetInt(int &i){
    i = 5;
}
void main(){
    int x;
    SetInt(x);
}

```

Cele două secvențe de cod fac același lucru, la prima vedere părând că nu există diferențe între conceptul de pointer și cel de referință.

Totuși, există unele **diferențe între pointeri și referințe**:

1. Pointerii pot să rămână neinițializați, în timp ce referințele trebuie inițializate.

```

int i = 10; int *p = &i; int &ref = i;
int j = 20; int *p1;
int &ref1; // eroare la compilare - referința nu a fost inițializată

```

2. Pointerii își pot schimba valoarea în timpul execuției programului, referințele rămân doar cu valoarea cu care au fost inițializate.

```

int i =10; int j = 20;
int *p = &i; p = &j;
int &ref = i;
&ref = j; // eroare la compilare - referința nu își poate schimba valoarea

```

3. Pointerii acceptă operații aritmetice (+, -, ++). Referințele nu acceptă nici o operație aritmetică.

```

int i = 10; int j = 20;
int *p = &i; //pointer către i
p++; //pointerul se mută la variabila j
(*p) = 30;
cout<< j; //30
int &ref = i;
ref++; // i va avea valoarea 11
(&ref)++; //eroare la compilare - referința nu suportă operații aritmetice

```

4. Pointerii pot fi convertiți ("castați") la alte tipuri de pointer (orice pointer poate fi convertit la `void*`). Referințele nu pot fi convertite ("castate").

```

int i =10;
char *p = (char *)&i;
char &ref = i; // eroare la compilare, referința de tip char poate puncta doar la char

```

5. Există pointeri către pointeri, dar nu referințe către referințe.

```

int i =10; int *p = &i;
int **p_to_p = &p;
**p_to_p = 20; //valoarea lui i devine 20
int j = 10;
int &ref = j;
int & &ref_to_ref = ref; // eroare la compilare

```

6. Pot exista vectori de pointeri, dar nu vectori de referințe.

3.3.5. Pointeri la pointeri

Putem să avem pointer către o variabilă de tipul pointer, așa cum putem avea pointer către orice alt tip de variabilă.

```

int main(){
    int valoare =10;
    int *pointer = &valoare;
    int **pointer_la_pointer = &pointer;
    cout<< valoare; //10
    cout<< pointer; // adresa de memorie a variabilei valoare
}

```

```
cout<< *pointer; //10
cout<< pointer_la_pointer // adresa de memorie a pointerului pointer
cout<< *pointer_la_pointer // adresa de memorie a variabilei valoare
cout<< **pointer_la_pointer; //10
```

3.3.6. Pointeri la funcții

În C un pointer referențiază o adresă de memorie. Astfel, pe lângă pointerii normali (cei care duc la adresa unor variabile) putem avea și pointeri către o funcție (pointerul duce la adresa începutului de cod a funcției). Un pointer către o funcție este declarat în felul următor:

```
tip_returnat (*nume_funcție)(tip parametri)
```

După ce definim un pointer către o funcție va trebui să îi asignăm o funcție. Exemplu de program care folosește pointeri către funcții:

```
#include <stdio.h>
#include <iostream>
int suma(int x, int y){
    return x+y;
}
int main(){
    int (*pointer_funcție)(int,int);
    pointer_funcție = suma;
    int s1 = pointer_funcție(10,15);
    int s2 = suma(10,15);
    cout<<s1<<" "<<s2; // 25 25
    return 0;
}
```

3.4. Crearea tipurilor de date

Limbajul C permite crearea tipurilor de date în 5 moduri:

- **structura (struct)** – grupează mai multe variabile sub același nume, creează un nou tip de date agregând tipuri diferite de date;
- **câmpul de biți** (variațiune a structurii) - acces ușor la biții individuali;
- **uniunea (union)** – face posibil ca aceleași zone de memorie să fie definite ca două sau mai multe tipuri diferite de variabile;
- **enumerarea (enum)** – listă de constante întregi cu nume;
- **tipuri definite de utilizator (typedef)** – definește un nou nume pentru un tip existent.

3.4.1. Structuri

□ variabile grupate sub același nume.

□ sintaxa:

```
struct <nume> {  
    < tip 1 >    <variabila 1>;  
    < tip 2 >    <variabila 2>;  
    -----  
    < tip n >    <variabila n>;  
} lista_identificatori_de_tip_struct;
```

□ variabilele care fac parte din structură sunt denumite membri (elemente sau câmpuri) ai structurii.

□ **struct** <nume> {
 < tip 1 > <variabila 1>;
 < tip 2 > <variabila 2>;

 < tip n > <variabila n>;
} lista_identificatori_de_tip_struct;

□ observații:

- dacă numele structurii (<nume>) lipsește, structura se numește anonimă. Dacă lista identificatorilor declarați lipsește, se definește doar tipul structură. Cel puțin una dintre aceste specificații trebuie să existe.
- dacă <nume> este prezent → se pot declara noi variabile de tip structura **struct <nume> <lista noilor identificatori>;**
- referirea unui membru al unei variabile de tip structură se face cu operatorul de selecție punct . care precizează identificatorul variabilei și al câmpului.

□ exemplu:

```
struct student {  
    char nume[30];  
    char prenume[30];  
    float notaExamen;  
} A, B, C;  
Definește un tip de structură numit  
student și declară ca fiind de acest  
tip variabilele A, B, C
```

```
struct {  
    char nume[30];  
    char prenume[30];  
    float notaExamen;  
} A;  
Declară o variabilă numită A  
definită de structura care o  
precede.
```

```
typedef struct {  
    char nume[30];  
    char prenume[30];  
    float notaExamen;  
} student;  
student A;  
Definește un tip de date numit student și  
declară ca fiind de acest tip variabila A
```

Inițializarea structurilor

□ exemplu:

```
student.c  
1  #include<stdio.h>  
2  
3  
4  typedef struct {  
5      char nume[30];  
6      char prenume[30];  
7      float notaExamen;  
8  } student;  
9  
10 int main()  
11 {  
12     student A = {"Popescu","Maria",9.25};  
13     student B = {.notaExamen = 9.25,.prenume = "Maria",.nume = "Popescu"};  
14     student C = {"Popescu"};  
15  
16     printf("%s %s %f \n", A.nume,A.prenume,A.notaExamen);  
17     printf("%s %s %f \n", B.nume,B.prenume,B.notaExamen);  
18     printf("%s %s %f \n", C.nume,C.prenume,C.notaExamen);  
19  
20     return 0;  
21 }
```

```
typedef struct {  
    char nume[30];  
    char prenume[30];  
    float notaExamen;  
} student;
```

```
Bogdan-Alexes-MacBook-Pro:curs4 bogdan$ gcc student.c  
Bogdan-Alexes-MacBook-Pro:curs4 bogdan$ ./a.out  
Popescu Maria 9.250000  
Popescu Maria 9.250000  
Popescu 0.000000
```

Structuri imbricate și tablouri de structuri

- ❑ o structură este imbricată (nested) dacă ea conține ca membru o altă structură

```
struct student{
    char nume[30];
    char prenume[30];
    float notaExamen;
    struct adresa;
}
```

Structura `adresa` trebuie să fie definită în prealabil

- ❑ tablou de structuri: tablou cu elemente de tip struct
`struct student grupa[30];`

Operații permise cu structuri

- accesul la membri structurii se face prin folosirea operatorului punct: `nume_variabila.nume_camp`
- atribuire pentru variabile de tip structură: `B = A;`
- obținerea adresei unei variabile structură
- utilizarea operatorului `sizeof` pentru determinarea dimensiunii unei structuri

Definirea unei structuri nu ocupă memorie ci doar creează un tip nou de date

- variabile declarate de tipul structurii respective ocupă memorie dimensiunea memoriei ocupată de o astfel de variabilă este aproximativ suma dimensiunilor de memorie ocupată de fiecare componentă
- zona de memorie ocupată este în final aliniată (se introduc, dacă este necesar, octeți în plus – padding bytes) pentru a facilita accesul la date (citire)

3.4.2. Câmpuri de biți

Un câmp de biți este un tip special de membru al unei structuri, care definește cât de lung (mare) trebuie să fie acel câmp, în biți. Astfel, putem stoca mai multe variabile într-un singur octet. Nu se poate obține adresa unui câmp de biți. Folosirea câmpurilor de biți este foarte utilă în economisirea memoriei utilizate.

▼ Sintaxa

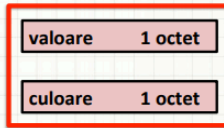
- ❑ **Sintaxa:**

```
struct <nume> {
    <tip 1> <variabila 1>: lungime;
    <tip 2> <variabila 2>: lungime;
    -----
    <tip n> <variabila n>: lungime;
} lista_identificatori_de_tip_struct;
```
- ❑ **Observații:**
 - ❑ tipul câmpului de biți poate fi doar întreg: `char`, `int`, `unsigned` sau `signed`.
 - ❑ câmpul de biți cu lungimea 1 → `unsigned` (un singur bit nu poate avea semn).
 - ❑ unele compilatoare → `doar unsigned`.
 - ❑ lungime → numărul de biți dintr-un câmp.

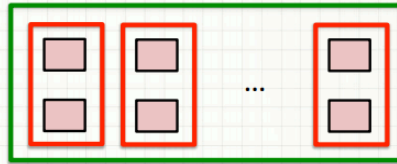
▼ Exemplul 1

- exemplu: în cadrul unui joc de cărți reprezentăm o carte printr-o structură

```
struct carteJoc {
    unsigned char valoare; // valori între 1 și 13
    unsigned char culoare; // valori între 1 și 4
} A;
```

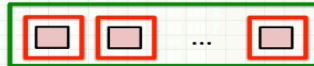


```
struct carteJoc PachetCartiJoc[52];
```



dimensiune A este 2
dimensiune pachetCartiJoc este 104

```
struct carteJoc {
    unsigned char valoare : 4; // 4 biti
    unsigned char culoare : 2; // 2 biti
} A;
struct carteJoc PachetCartiJoc[52];
```



dimensiune A este 1
dimensiune pachetCartiJoc este 52

▼ Exemplul 2

- exemplu: în aceeași structură putem combina câmpuri de biți și membri normali

```
struct adrese {
    char nume[30];
    char strada[40];
    char oras[20], jud[3];
    int cod;
};
```

```
struct angajat {
    struct adrese A;
    float plata;
    unsigned statut: 1; // activ sau intrerupt
    unsigned plata_ora: 1; // plata cu ora
    unsigned impozit: 3; // impozit rezultat
};
```

- observație:** definește o înregistrare despre salariat care folosește doar un octet pentru a păstra 3 informații: statutul, dacă este platit cu ora și impozitul.
- fără câmpul de biți, aceste informații ar fi ocupat 3 octeți.

3.4.3. Uniuni

Uniunea este un tip special de structură, ai cărei membri folosesc, la momente diferite, aceeași locație de memorie. De obicei, membrii unei uniuni au tipuri diferite.

▼ Sintaxa

- tip special de structuri ai cărei membri folosesc la momente diferite aceeași locație în memorie.
- membri unei uniuni au de obicei tipuri diferite

Sintaxa:

```
union <nume> {
    < tip 1 >    <variabila 1>;
    < tip 2 >    <variabila 2>;
    -----
    < tip n >    <variabila n>;
} lista_identificatori_tip_uniun;
```

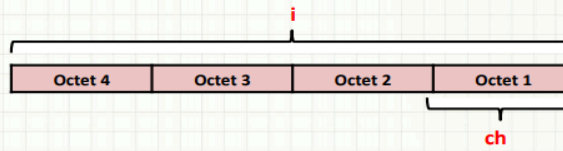
▼ Exemplul 1

- ❑ tip special de structuri ai cărei membri folosesc la momente diferite aceeași locație în memorie.
- ❑ membri unei uniuni au de obicei tipuri diferite

❑ exemplu:

```
union tip_u {
    int i;
    char ch;
};
```

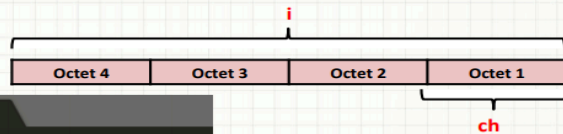
```
union tip_u A;
```



- ❑ când este declarată o variabilă de tip **union** compilatorul alocă automat memorie suficientă pentru a păstra cel mai mare membru al acesteia.

▼ Exemplul 2

❑ exemplu



```
union.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main()
6 {
7     union tip_u{
8         int i;
9         unsigned char ch;
10    };
11
12    union tip_u A;
13    A.i = 0;
14    printf("Dimensiunea lui A este %lu \n", sizeof(A));
15    A.ch = 1;
16    printf("A.ch = %d\n", A.ch);
17    printf("A.i = %d\n", A.i);
18
19    A.i = 300;
20    printf("A.ch = %d\n", A.ch);
21    printf("A.i = %d\n", A.i);
22
23    return 0;
24 }
```

```
Dimensiunea lui A este 4
A.ch = 1
A.i = 1
A.ch = 44
A.i = 300
```